QALight

Frontend-розробка

Заняття 11. Концепція семантичної розмітки

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Концепція семантичної розмітки

Семантичні елементи (section, nav, footer, header, article, main, aside)

Розробка унікального стилю для власного сайта.

Концепція семантичної розмітки

Семантичні елементи (section, nav, footer, header, article, main, aside)

Концепція семантичної розмітки

Теорія: Концепція семантичної розмітки

Концепція семантичної розмітки - це підхід до створення вебсторінок, за якого вибір тегів HTML базується на значенні та ролі контенту, а не на його зовнішньому вигляді. Сама по собі семантична розмітка створюється за допомогою HTML5, але вона тісно пов'язана з CSS. HTML визначає, чим є елемент (наприклад, заголовок, стаття, навігація), а CSS - як він виглядає (колір, шрифти, розташування).

Семантика проти Не-семантики

До появи HTML5 розробники часто використовували універсальні теги та для всього підряд, додаючи їм класи для стилізації в CSS.

Семантичний підхід замінив цей хаос чіткою структурою.

Несемантичний код: , , (Теги div та span нічого не говорять браузеру про свій вміст).

Семантичний код: <header>, <footer>, <h1> (Теги чітко вказують на свою функцію).

Основні семантичні теги та їхня роль

Теорія: Концепція семантичної розмітки

- Тег <header>: Шапка сайту або окремої секції (логотип, назва, пошук).
- Тег <nav>: Навігаційна панель із посиланнями на інші сторінки або розділи.
- Тег <main>: Основний, унікальний контент сторінки (може бути лише один на сторінці).
- Тег <section>: Тематичний блок, який об'єднує схожий контент (наприклад, "Про нас", "Послуги").
- Тег <article>: Самостійний, незалежний розділ контенту (пост у блозі, новина, картка товару).
- Тег <aside>: Бічна панель (сайдбар) з додатковим контентом (реклама, посилання на схожі статті).
- Тег <footer>: Підвал сайту або секції (авторські права, контакти, юридична інформація).

Як семантика взаємодіє з CSS

Теорія: Концепція семантичної розмітки

Замість створення безлічі класів типу .main-navigation-menu-item, ви можете стилізувати семантичні теги безпосередньо або через прості селектори:

```
/* Стилізація головної навігації */
nav {
 display: flex;
 justify-content: space-between;
}

/* Стилізація бічної панелі */
aside {
 background-color: #f4f4f4;
 padding: 20px;
}
```

Як семантика взаємодіє з CSS

Теорія: Концепція семантичної розмітки

Замість створення безлічі класів типу .main-navigation-menu-item, ви можете стилізувати семантичні теги безпосередньо або через прості селектори:

```
/* Стилізація головної навігації */
nav {
 display: flex;
 justify-content: space-between;
}
/* Стилізація бічної панелі */
aside {
 background-color: #f4f4f4;
 padding: 20px;
}
```

Коли структура HTML зрозуміла, легше будувати архітектуру CSS (наприклад, за методологією БЕМ), де семантичний тег виступає логічним блоком. Ви можете легко змінити візуальне відображення елемента в CSS, але його значення для пошукових роботів залишиться незмінним. Наприклад, зробити посилання <a href="..."> візуально схожим на кнопку за допомогою CSS, але для екранних читалок воно все одно залишиться посиланням.

Методологія БЕМ (Блок, Елемент, Модифікатор)

Теорія: Концепція семантичної розмітки

Методологія БЕМ (Блок, Елемент, Модифікатор) — це популярний підхід до організації CSS-класів та створення зручних для підтримки веб-інтерфейсів. Її розробили в компанії «Яндекс» для великих і складних проєктів. [1] (https://ua.linkedin.com/pulse/bem-block-element-modifier-methodology-organizing-your-palak-soni-ciukf?tl=uk), [2] (https://bem.info/ru/methodology/quick-start/), [3] (https://github.com/dandelionadia/docs/blob/master/BEM/BEM.md)Основні поняттяБлок (Block):

Самостійний і незалежний компонент інтерфейсу (наприклад, кнопка — button, або меню — menu).Елемент (Element): Частина блоку, яка не може існувати окремо від нього (наприклад, пункт меню — menu__item). Обозначається двома підкресленнями __.Модифікатор (Modifier): Знак або стан блоку чи елемента (наприклад, активна кнопка — button--active). Обозначається двома дефісами --.Головні перевагиМасштабність: Код легко розширювати без ризику зламати інші стилі.Читабельність: Назви класів одразу показують структуру та зв'язки в коді.Повторне використання: Блоки можна переносити на інші сторінки або в інші проєкти.Якщо хочете, розкажіть:Який проєкт ви зараз розробляєте (сайт-візитка, великий веб-додаток)?Чи потрібні вам приклади написання коду за цією системою?Я допоможу розібратися детальніше.

Головні правила БЕМ

Теорія: Концепція семантичної розмітки

Головні правила БЕМ, які часто порушують

- Немає каскаду: Елементи не повинні містити назв інших елементів (наприклад, product-card__title__text - це помилка). Правильно робити плоску структуру: product-card__text.
- Модифікатор не живе сам по собі: Не можна писати клас модифікатора без основного класу (наприклад, просто <button class="product-card__button--disabled"> - помилка).
- Модифікатор завжди додається поруч з основним класом.
- Незалежність: Блоки не повинні залежати від інших блоків на сторінці. Їх можна легко перенести в будь-яке інше місце сайту.

Приклад компонента картки товару

Теорія: Концепція семантичної розмітки

```
<!-- Блок -->
<div class="product-card">
 <!-- Елемент -->
 <img src="photo.jpg" alt="Товар" class="product-card__image">
  <!-- Елемент -->
 <h3 class="product-card__title">Смартфон Galaxy</h3>
  <!-- Елемент з Модифікатором (акційна ціна) -->
 <span class="product-card__price product-card__price--discount">$799</span>
  <!-- Елемент-кнопка з Модифікатором (заблокована) -->
 <button class="product-card__button product-card__button--disabled" disabled>
   Немає в наявності
 </button>
</div>
```

Приклад компонента картки товару (стилі)

Теорія: Концепція семантичної розмітки

```
/* Блок */
.product-card {
 border: 1px solid #ccc;
 padding: 16px;

 /* Елемент */
 &__image {
   width: 100%;
 }

 /* Елемент */
 &__title {
   font-size: 18px;
   margin-bottom: 8px;
 }
```

Приклад компонента картки товару (стилі)

Теорія: Концепція семантичної розмітки

```
/* Елемент */
&__price {
  font-weight: bold;

  /* Модифікатор */
  &--discount {
    color: red;
  }
}
/* Елемент */
&__button {
  background-color: blue;
  /* Модифікатор */
  &--disabled {
    background-color: gray;
  }
}
```

Чому семантична розмітка є критично важливою?

Теорія: Концепція семантичної розмітки

Доступність (Accessibility / A11y):

Екранні читалки (screen readers) для людей з порушеннями зору використовують семантичні теги, щоб озвучувати структуру сайту. Користувач може одразу "перестрибнути" до розділу <nav> або <main>.

SEO-оптимізація:

Пошукові роботи (Google) значно краще розуміють архітектуру сайту та індексують контент, якщо він лежить у правильних тегах (наприклад, текст у <article> має більшу вагу, ніж у звичайному).

Підтримка коду:

Іншим розробникам (і вам через кілька місяців) набагато простіше читати та редагувати HTML-код, який має чітку структуру, а не складається з "дивового супу" (div soup).

Модуль CSS Grid Layout

Теорія: Концепція семантичної розмітки

Модуль CSS Grid Layout - це система **двовимірного** макета, що оптимізована для дизайну інтерфейсу користувача. Головна ідея, що лежить в основі макету сітки, полягає в **поділі веб-сторінки на стовпці та рядки**. В області сітки, що утворилися, можна поміщати елементи сітки, а керувати їх розмірами і розташуванням можна за допомогою спеціальних властивостей модуля.

Спочатку потрібно визначити елемент контейнера з **display:grid**, встановити **розміри стовпців і рядків** за допомогою **grid-template-columns і grid-template-rows**, а потім розмістити його дочірні елементи за допомогою **grid-column. і grid-row**.

Вихідний порядок елементів **grid** не має значення. CSS може розміщувати їх у будь-якому порядку, що спрощує зміну порядку grid за допомогою медіа-запитів.

Прклади шаблонів.

Контейнер Grid

Теорія: Концепція семантичної розмітки

Для створення макету на основі **grid** необхідно визначити **grid-контейнер**. Контейнер-сітка (**grid container**) - це блок, який створює область з сіткою, а дочірні елементи розташовуються відповідно до правил компонування сітки. Коли ви визначаєте контейнер сітки за допомогою display: **grid** або display: **inline-grid**, ви створюєте новий **контекст форматування вмісту** цього контейнера, який **впливає тільки на дочірні елементи сітки**.

Контейнер grid - це елемент HTML, властивість display якого є **grid** або **inline-grid**.

Елемент grid - це будь-який із прямих дочірніх елементів контейнера grid. Елемент, до якого застосовується display:grid - прямий батько всіх елементів сітки.

```
<div class="grid-container">
    <div><img src="images/cat1.jpg"></div>
    <div><img src="images/cat2.jpg"></div>
    <div><img src="images/cat3.jpg"></div>
    <div><img src="images/cat4.jpg"></div>
    <div><img src="images/cat5.jpg"></div>
    <div><img src="images/cat1.jpg"></div>
</div>
```

Концепція сітки

Теорія: Концепція семантичної розмітки

Концепція grid являє собою набір горизонтальних і вертикальних ліній, що перетинаються, ділять простір grid-контейнера на **області сітки**, в які можуть бути поміщені елементи сітки. Лінії сітки (**grid lines**) - це невидимі горизонтальні та вертикальні розділові лінії, вони існують по обидва боки від рядка та стовпця. На них можна посилатися за числовим індексом (використовуючи властивості grid-column-start, grid-column-end, grid-row-start та grid-row-end) або імені, заданому в CSS-коді.

Виділяють дві групи ліній сітки: одна група визначає **стовпці**, які проходять вздовж осі блоку (**вісь стовпців**), і перпендикулярна група, що визначає **рядки**, що тягнуться вздовж лінійної осі (**вісь рядків**). Доріжка сітки (**grid track**) - простір між двома сусідніми лініями сітки, що використовується для визначення стовпця або рядка сітки. Доріжка йде від краю контейнера до іншого, розмір залежить від розташування ліній сітки. Доріжки сітки аналогічні стовпцям та рядкам таблиці. Суміжні доріжки щільно прилягають одна до одної, встановити відстань між ними можна за допомогою властивостей **row-gap**, **column-gap** і **gap**.

Визначення сітки

Теорія: Концепція семантичної розмітки

При створенні **grid-**контейнера стандартна сітка має один стовпець і один рядок, які займають повний розмір контейнера. Для поділу контейнера-сітки на стовпці та рядки використовуються властивості **grid-template-columns, grid-template-rows та grid-template-areas**.

Остаточна сітка може виявитися більшою через елементи сітки, розміщені поза сіткою; у цьому випадку будуть створені неявні доріжки, розмір цих неявних доріжок визначатиметься властивостями grid-auto-rows та grid-auto-columns.

Властивості grid та grid-template – це скорочені позначення, які можна використовувати для одночасної установки всіх трьох явних властивостей сітки grid-template-columns, grid-template-rows та grid-template-areas. grid скидає властивості, що управляють неявною сіткою, тоді як властивість grid-template залишає їх без змін.

Стовпці та рядки сітки

Теорія: Концепція семантичної розмітки

Кількість рядків / стовпців визначається за допомогою властивостей **grid-template-rows та grid-template-columns**. Властивості не успадковуються.

Значення **none** (за замовчуванням) вказує, що властивість не створює явних доріжок сітки. За відсутності явної сітки будь-які рядки/стовпці генеруватимуться неявно, а їх розмір визначатиметься властивостями grid-auto-rows та grid-auto-columns.

Список доріжок встановлює список доріжок у вигляді послідовності функцій визначення розміру доріжок та назв ліній сітки. Розміри доріжок сітки задають за допомогою позитивних значень, використовуючи відносні одиниці довжини, em, vh, vw; абсолютні одиниці довжини - px; та відсотки %. Розміри % обчислюються від ширини або висоти контейнера-сітки.

Властивість **grid-template-columns** використовується для визначення розміру та кількості стовпців в макеті сітки. `grid-template-columns: 100px 150px auto 60px;`

Властивість **grid-template-rows** використовується для визначення розміру та кількості рядків в макеті сітки. `grid-template-rows: 100px 200px 300px;`

Гнучкі розміри доріжок

Теорія: Концепція семантичної розмітки

Одиниця виміру **fr** - одиниця довжини, яка дозволяє створювати гнучкі доріжки. Не є одиницею виміру у звичайному її розумінні, тому може бути представлена чи об'єднана з іншими типами одиниць у виразах **calc()**. Загальний розмір фіксованих рядків або стовпців віднімається з доступного простору контейнера-сітки. Простір, що залишився, ділиться між рядками і стовпцями з гнучкими розмірами пропорційно їх коефіцієнту, наприклад:

```
.grid-container {
        display: grid;
        grid-template-columns: 1fr 1fr 1fr 1fr; /* эквивалентно grid-template-columns: 25% 25% 25% 25%; */
}
 .grid-container {
        display: grid;
        grid-template-columns: 1fr 2fr 1fr; /* эквивалентно grid-template-columns: 25% 50% 25%; */
}
```

Якщо сума всіх гнучких розмірів доріжок менше 1, вони займатимуть лише відповідну частку простору, що залишився, а не розширюватися, щоб заповнити його повністю. Якщо доступний простір нескінченний (тобто ширина чи висота контейнера-сітки не задані), доріжки сітки гнучкого розміру масштабуються за своїм вмістом, зберігаючи при цьому їх відповідні пропорції.

Мінімальні та максимальні розміри доріжок

Ключове слово **max-content** встановлює максимально необхідний простір для доріжки з урахуванням вмісту сітки.

min-content встановлює мінімальний простір, необхідний для вмісту, при цьому ширина елемента орієнтується на найдовше слово або найширше зображення.

Функція **minmax(min,max)** визначає діапазон розмірів, більший або рівний min і менший або рівний max. Якщо max < min, max ігнорується, а minmax(min,max) обробляється як min. **Значення у fr можна встановлювати лише як максимальне**.

```
.grid-container {
    display: grid;
    grid-template-rows: 200px minmax(100px, 1fr);
}
```

Автоматичні розміри

Теорія: Концепція семантичної розмітки

Значення **auto** орієнтується на вміст елементів сітки однієї доріжки. Рахується як мінімальний розмір елемента сітки, якщо визначено min-width або min-height.

Максимум обробляється так само, як і max-content. Може розтягуватися за рахунок властивостей align-content та justify-content.

```
.grid-container {
 display: grid;
 grid-template-rows: auto 1fr;
 grid-template-columns: auto 1fr auto;
}
```

Відповідність вмісту

Теорія: Концепція семантичної розмітки

Розміри доріжок можна задавати за допомогою значення **fit-content**(довжина або %), що є формулою **min(maximum size, max(minimum size, argument))**, яка обчислюється як **minmax(auto, max-content)**, тобто **auto**. При цьому розмір доріжки обмежується значенням, вказаним у дужках, якщо воно більше, ніж автоматичний мінімум.

```
.grid-container {
 display: grid;
 grid-template-columns: fit-content(50%) fit-content(300px) 1fr;
}
```

Повтор рядків та стовпців

Теорія: Концепція семантичної розмітки

Нотація **repeat()** - це фрагмент списку доріжок, що повторюється. repeat дозволяє записати в більш компактній формі кілька однакових за розмірами стовпців або рядків. repeat() має кілька форм:

/*повтор доріжки*/
repeat(кількість повторень, ім'я доріжки? розмір доріжки + ім'я доріжки?)
/*автозаповнення доріжок сітки*/
repeat(auto-fill або auto-fit, ім'я доріжки? фіксований розмір доріжки + ім'я доріжки?)
/*фіксований повтор доріжки*/
repeat(кількість повторень, ім'я доріжки? фіксований розмір доріжки + ім'я доріжки?)

Значення **auto-fill** створює стільки доріжок, скільки потрібно для заповнення контейнера сітки, навіть якщо деякі з них порожні.
Автопідгонка - **auto-fit**: створює стільки доріжок, скільки потрібно, але якщо деякі доріжки порожні, auto-fit згортає їх, щоб вони не займали місця.

Нотація repeat() може бути вкладеною. Значення auto-fill або auto-fit не сумісні з min-content, max-content, auto, fit-content() або fr.

Іменовані області сітки

Теорія: Концепція семантичної розмітки

Grid-template-areas визначає шаблон сітки, посилаючись на назви областей сітки, визначені властивістю **grid-area**. Повторення назви області сітки призводить до того, що вміст охоплює ці клітинки. Крапка означає порожню клітинку .

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr 1fr;
  grid-template-rows: auto;
  grid-template-areas:
    "header header header header"
    "main main . aside"
    "footer footer footer footer";
}
```

Це створить сітку з чотирьох стовпців та трьох рядків. Верхній рядок буде складатися з області заголовка. Середній рядок складатиметься з двох основних областей, однієї порожньої клітинки та однієї бічної панелі. Останній рядок - це нижній колонтитул.

\

Використання grid-area

Теорія: Концепція семантичної розмітки

`grid-area` – назва області сітки, визначеної за допомогою **grid-template-areas**

```
header {
 grid-area: header;
}
aside {
 grid-area: sidebar;
}
main {
 grid-area: content;
}
```

Властивість grid-template-columns

Теорія: Концепція семантичної розмітки

Властивість grid-template-columns використовується для визначення кількості та ширини стовпців у сітці.

```
.grid-container {
    display: grid;
    grid-template-columns: repeat(3,1fr);
  }
```

Наступний код CSS встановлює властивість grid-column для охоплення від першого стовпця до третього стовпця.

```
.box1 {
   grid-column: 1/3;
}
```

Властивість grid-template-rows

Теорія: Концепція семантичної розмітки

Властивість grid-template-rows визначає розмір і кількість рядків у сітці. Тут властивість grid-template-rows повторюється 3 рази, при цьому кожен рядок має висоту 100 пікселів. Це означає, що сітка матиме загалом 3 рядки, кожна з яких має висоту 100 пікселів.

```
.grid-container {
   display: grid;
   grid-template-rows: repeat(3,100px);
 }
```

Наступний код CSS встановлює властивість grid-row для охоплення від першого рядка до третього рядка.

```
.box1 {
   grid-column: 1/3;
   grid-row: 1/3;
}
```

Властивість grid-template

Теорія: Концепція семантичної розмітки

Властивість grid-template використовується для визначення кількості та розміру доріжок сітки (рядків і стовпців) у контейнері сітки. Властивість **grid-template** є скороченням для встановлення grid-template-rows, grid-template-columns та grid-template-areas в одному оголошенні.

```
.grid-container {
  display: grid;
  grid-template: repeat(3, 200px)/repeat(3, 1fr);
}
```

Наступний код вказує, що елемент має починатися в рядку 1 і закінчуватися в рядку 3, а також починатися в стовпці 1 і закінчуватися в стовпці 3. Це означає, що елемент охоплюватиме перші два рядки та перші два стовпці сітки.

```
.box1 {
  grid-row-start: 1;
  grid-row-end: 3;
  grid-column-start: 1;
  grid-column-end: 3;
}
```

Розташування стовпців контейнера

Теорія: Концепція семантичної розмітки

justify-content визначає, як браузери мають розташувати стовпці контейнера вздовж осі рядка. Вісь рядка іноді називають внутрішньою віссю. Властивість justify-content працює, якщо загальна ширина стовпця менша за ширину контейнера сітки. Іншими словами, вам потрібен вільний простір уздовж осі рядків контейнера, щоб вирівняти його стовпці ліворуч або праворуч.

Властивість justify-content приймає такі значення:

- start розміщує стовпці контейнера сітки з його початковим краєм рядка.
- center розміщує стовпці контейнера сітки по центру осі рядків сітки.
- end розміщує стовпці контейнера сітки з його краєм кінця рядка.
- stretch розтягує елементи контейнера сітки, щоб заповнити вісь рядків
- space-between створює рівномірний відстань між кожною парою стовпців між першим і останнім стовпчиком сітки.
- space-around призначає однакові відстані для кожної сторони стовпців контейнера сітки.
- space-evenly призначає рівномірну відстань до обох кінців контейнера сітки та між його стовпцями.

Вирівнювання по осі

Теорія: Концепція семантичної розмітки

Елементи сітки можуть бути вирівняні у напрямку осі рядка за допомогою властивості **justify-self** або властивості **justify-items** (заданого для контейнера-сітки).
Властивість justify-items визначає стандартне значення justify-self для всіх елементів сітки.
Властивість justify-items приймає такі значення:

- stretch - це значення за замовчуванням для вирівнювання елементів - розтягує елементи контейнера сітки, щоб заповнити вісь рядків (внутрішню).
- start розміщує елементи контейнера сітки з краю рядка осі рядка.
- center розташовує елементи контейнера сітки по центру осі рядків.
- end розміщує елементи контейнера сітки з кінця рядка осі рядка.

Властивість **align-items** визначає стандартне значення **align-self** для всіх елементів сітки.
Властивість align-items приймає такі значення:

- stretch - значення за замовчуванням для align-items - розтягує елементи контейнера сітки, щоб заповнити осі стовпців (блоків).
- start вирівнює елементи контейнера сітки з початку стовпця осі стовпців.
- center вирівнює елементи контейнера сітки по центру осі стовпців.
- end вирівнює елементи контейнера сітки з кінця стовпця осі стовпців.

Проміжки між елементами сітки

Теорія: Концепція семантичної розмітки

Властивості row-gap та column-gap (і їх скорочений запис gap) визначають проміжки між рядками та стовпцями сітки. Кожен проміжок розглядається як додаткова порожня доріжка зазначеного розміру. Проміжки додаються лише між двома доріжками сітки, тобто вони не додаються перед першою та після останньої доріжки.

```
.container {
  grid-template-columns: 100px 50px 100px;
  grid-template-rows: 80px auto 80px;
  column-gap: 10px;
  row-gap: 15px;
}
```

Відсоткове значення обчислюється щодо розмірів області сітки. Негативні значення не використовуються.

gap: 1%;

Скорочення для row-gap і column-gap: gap: 15px 10px;

Приклад побудови класичного макета сайту

Теорія: Концепція семантичної розмітки

```
<header class="site-header">
  <h1>Мій Логотип</h1>
  <nav class="site-nav">
    <ul>
      <li><a href="#">Головна</a></li>
      <li><a href="#">Блог</a></li>
      <li><a href="#">Контакти</a></li>
    </ul>
  </nav>
</header>
<main class="main-content">
  <article class="post">
    <h2>Заголовок цікавої статті</h2>
    <p>Це основний контент сторінки. Оскільки це окрема стаття, ми загорнули її в тег article всередині головного контейнера main.</p>
  </article>
</main>
<aside class="sidebar">
  <h3>Додатково</h3>
  <p>Бічна панель (сайдбар) із віджетами, рекламою або архівом статей.</p>
</aside>
<footer class="site-footer">
  <p>&copy; 2026 Усі права захищено.</p>
</footer>
```

Стилізація через CSS Grid

Теорія: Концепція семантичної розмітки

```css
/* Скидання базових відступів */
* {
 margin: 0;
 padding: 0;
 box-sizing: border-box;
}

body {
 font-family: Arial, sans-serif;
 line-height: 1.6;
 min-height: 100vh;
```

Перетворюємо body на Grid-контейнер

Теорія: Концепція семантичної розмітки

```
 /* Перетворюємо body на Grid-контейнер */
 display: grid;
 grid-template-columns: 3fr 1fr; /* Основний контент ширший, сайдбар вужчий */
 grid-template-rows: auto 1fr auto; /* Шапка і підвал по контенту, середина займає все місце */
  /* Описуємо карту макета */
 grid-template-areas:
   "header  header"  /* Шапка на всю ширину (2 колонки) */
   "main    sidebar" /* Основна частина та сайдбар поруч */
   "footer  footer"; /* Підвал на всю ширину (2 колонки) */
  gap: 15px; /* Відступи між блоками */
}
```

Прив'язка тегів до областей сітки

Теорія: Концепція семантичної розмітки

```
.site-header {
 grid-area: header;
 background-color: #2c3e50;
 color: white;
 padding: 20px;
 display: flex;
 justify-content: space-between;
 align-items: center;
}
.site-nav ul {
 display: flex;
 list-style: none;
 gap: 20px;
}
.site-nav a {
 color: white;
 text-decoration: none;
}
```

Прив'язка тегів до областей сітки

Теорія: Концепція семантичної розмітки

```
.main-content {
 grid-area: main;
 background-color: #ecf0f1;
 padding: 20px;
}
.sidebar {
 grid-area: sidebar;
 background-color: #e0e0e0;
 padding: 20px;
}
.site-footer {
 grid-area: footer;
 background-color: #34495e;
 color: white;
 text-align: center;
 padding: 15px;
}
```

Адаптивність (Мобільна версія)

Теорія: Концепція семантичної розмітки

```
/* Адаптивність (Мобільна версія) */
@media (max-width: 768px) {
 body {
   /* На смартфонах перебудовуємо все в одну колонку */
   grid-template-columns: 1fr;
   grid-template-areas:
     "header"
     "main"
     "sidebar"
     "footer";
 }
  .site-header {
   flex-direction: column;
   gap: 10px;
 }
```

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Vs Code : https://code.visualstudio.com/

Webstorm: https://www.jetbrains.com/webstorm/

Web technology For developers https://developer.mozilla.org/en-US/docs/Web

CSS: https://developer.mozilla.org/en-US/docs/Web/CSS

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

Консорціум Всесвітнього павутиння https://www.w3.org/

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua